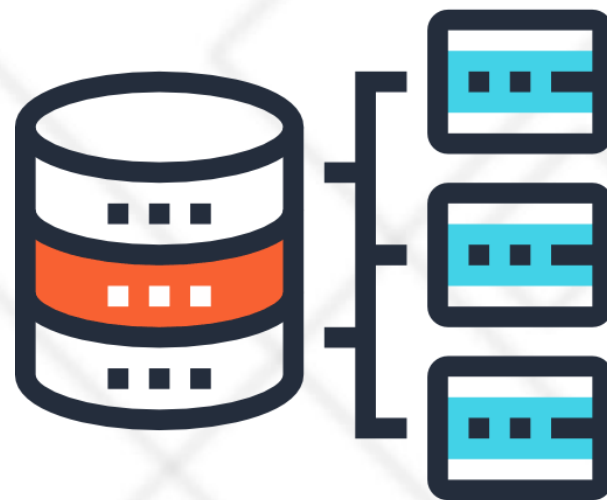




北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY



第3章 关系数据库

北京理工大学 计算机学院

段春晖

duanch@bit.edu.cn

第3章 关系数据库

- 关系数据库系统是支持关系模型的数据库系统
- 关系模型的组成
 - 数据模型通常由数据结构、数据操作和完整性约束三个要素组成
 - 关系数据结构、关系操作集合、关系完整性约束
- 关系数据库是以集合论中关系的概念为基础发展起来的，它运用数学方法研究数据库的结构和定义对数据的操作
- 本章将详细论述关系数据库涉及的基本概念及对关系的各种运算

第3章 关系数据库



□数据模型的三要素是本章学习的路标

- 静态数据结构；动态数据操作；完整性约束

□这三要素体现在如下问题中：

- 如何表达数据本身？
- 如何表达数据间的联系？
- 有哪些动态数据操作？
- 能表达哪些完整性约束？
- 完整性约束如何设定？
- 完整性约束的效果？
- 如何进行关系模型的DB设计？



第3章 关系数据库

- 3.1 关系模型的基本概念
- 3.2 关系代数
- 3.3 元组关系演算
- 3.4 域关系演算
- 3.5 小结

3.1 关系模型的基本概念

□3.1.1 关系的定义

- 1. 笛卡尔积 (Cartesian Product)
- 2. 域 (Domain)
- 3. 关系 (Relation)

□3.1.2 关系模式和关系数据库

□3.1.3 键

□3.1.4 完整性约束

3.1.1 关系的定义

□在关系理论是以集合代数理论为基础的，因此，我们可以用集合代数给出二维表的“关系”定义。先引入域和笛卡尔积的概念

□1. 域 (Domain)

- 域是一组具有相同数据类型的值的集合
 - ◆ 自然数、整数、{男、女}、{0、1}
- 关系中用域表示属性的取值范围，例如
 - ◆ $D1 = \{\text{李丽, 王平, 刘伟}\}$
 - ◆ $D2 = \{\text{男, 女}\}$
 - ◆ $D3 = \{47, 28, 30\}$
 - ◆ 其中 $D1$, $D2$, $D3$ 为域名，分别表示教师关系中姓名、性别、年龄的集合

2. 笛卡尔积

□ 1) 笛卡尔积 (Cartesian Product)

- 定义3.1 给定一组集合 $D_1, D_2, \dots, D_n$, 它们可以是相同的。 $D_1, D_2, \dots, D_n$ 的笛卡尔积为:

$$D_1 \times D_2 \times \dots \times D_n = \{(d_1, d_2, \dots, d_n) \mid d_i \in D_i, i=1, 2, \dots, n\}$$

- 所有域的所有值的一个组合, 不能重复

2. 笛卡尔积

[例] 给出域: $D_1 = \text{SUPERVISOR} = \{\text{张清玫}, \text{刘逸}\}$

$D_2 = \text{SPECIALITY} = \{\text{计算机专业}, \text{信息专业}\}$

$D_3 = \text{POSTGRADUATE} = \{\text{李勇}, \text{刘晨}, \text{王敏}\}$

则 $D_1 \times D_2 \times D_3 =$

$\{(\text{张清玫}, \text{计算机专业}, \text{李勇}), (\text{张清玫}, \text{计算机专业}, \text{刘晨}),$
 $(\text{张清玫}, \text{计算机专业}, \text{王敏}), (\text{张清玫}, \text{信息专业}, \text{李勇}),$
 $(\text{张清玫}, \text{信息专业}, \text{刘晨}), (\text{张清玫}, \text{信息专业}, \text{王敏}),$
 $(\text{刘逸}, \text{计算机专业}, \text{李勇}), (\text{刘逸}, \text{计算机专业}, \text{刘晨}),$
 $(\text{刘逸}, \text{计算机专业}, \text{王敏}), (\text{刘逸}, \text{信息专业}, \text{李勇}),$
 $(\text{刘逸}, \text{信息专业}, \text{刘晨}), (\text{刘逸}, \text{信息专业}, \text{王敏})\}$

2. 笛卡尔积

□2) 元组 (Tuple)

- 笛卡尔积中每一个元素 $(d_1, d_2, \dots, d_n)$ 叫做一个 n 元组 (n -tuple) 或简称元组
- 例: (张清玫, 计算机专业, 李勇)、(张清玫, 计算机专业, 刘晨) 是元组

□3) 分量 (Component)

- 笛卡尔积元素 $(d_1, d_2, \dots, d_n)$ 中的每一个值 d_i 叫做一个分量
- 例: 张清玫、计算机专业、李勇、刘晨是分量

3. 关系

□1) 关系

- **定义3.2** $D_1 \times D_2 \times \dots \times D_n$ 的任一个子集称为 $D_1, D_2, \dots, D_n$ 上的一个关系。 n 叫做关系的目或度 (degree)

□2) 元组和属性

- 关系中的每一行对应一个元组，通常用 t 表示
- 每一列对应一个域。关系中的列称为属性，每一列用属性名表示。 $t[A_i]$ 表示元组 t 在属性 A_i 上的值

□3) 一元关系与二元关系

- 当 $n=1$ 时，称该关系为一元关系 (unary relation)
- 当 $n=2$ 时，称该关系为二元关系 (binary relation)

3. 关系

□例如在上例的笛卡尔积中取出有实际意义的元组来构造关系

SAP (SUPERVISOR, SPECIALITY, POSTGRADUATE)

□假设

- 导师与专业：n:1（一个导师只能对一个专业）
- 导师与研究生：1:n（一个研究生只能遵从一个导师）

□于是：SAP关系可以包含三个元组

SUPERVISOR	SPECIALITY	POSTGRADUATE
张清玫	计算机专业	李勇
张清玫	计算机专业	刘晨
刘逸	信息专业	王敏

3. 关系

- 按照定义，关系可以是一个无限集合
- 由于笛卡尔积不满足交换律，所以
$$(d_1, \dots, d_i, d_j, \dots, d_n) \neq (d_1, \dots, d_j, d_i, \dots, d_n)$$
- 严格地说，**关系**是一种**规范化了的二维表中行的集合**，当关系作为关系数据模型的数据结构时，需要作出补充和限定
 - 无限关系在数据库中是无意义的，因此限定关系数据模型中的关系必须是有限集合
 - 通过为关系的每列增加一个属性名的方法取消元组的有序性，即
$$(d_1, \dots, d_i, d_j, \dots, d_n) = (d_1, \dots, d_j, d_i, \dots, d_n), (i, j=1, 2, \dots, n)$$

规范化的关系的性质

1) 列是同质的 (Homogeneous)

- 每一列中的分量是同一类型的数据，来自同一个域

2) 不同的列可出自同一个域

- 其中的每一列称为一个属性，不同的属性要给予不同的属性名

3) 列的顺序无所谓

- 列的次序可以任意交换
- 遵循这一性质的数据库产品（如ORACLE），增加新属性时，永远是插至最后一列
- 但也有许多关系数据库产品没有遵循这一性质，例如FoxPro仍然区分了属性顺序

规范化的关系的性质

4) 各个元组是不同的，即关系中不允许出现重复元组

- 有些数据库允许关系表中存在两个完全相同的元组

5) 行的顺序无所谓，即行的次序可以任意交换

- 遵循这一性质的数据库产品（如ORACLE），插入一个元组时永远插至最后一行
- 但也有许多关系数据库产品没有遵循这一性质，例如FoxPro仍然区分了元组的顺序

6) 分量必须取原子值

- 每一个分量都必须是不可分的数据项
- 这是规范条件中最基本的一条



3.1.2 关系模式和关系数据库

□ 关系模式 (Relation Schema) 是对关系的描述

- 该描述包括关系名、属性名、属性的类型和长度, 以及属性间固有的数据关联关系

- 关系模式一般简记为关系名和属性名的集合

$R(A_1, A_2, \dots, A_n)$, 或仅用关系名 R 表示

- 如图书关系模式可描述为:

图书(书号, 书名, 作者, 单价, 出版社)

□ 关系的值是元组的集合, 称为关系

- 关系是对现实世界中事物在某一时刻状态的反映, 关系的值是随时间在不断变化的

□ 关系模式和关系系统称关系, 通过上下文加以区别

3.1.2 关系模式和关系数据库

□ 关系数据库

- 关系模式的集合称为关系数据库模式，是对数据库中所有数据逻辑结构的描述，表示为

$$R = \{R_1, R_2, \dots, R_p\}$$

- 关系数据库模式中的每个关系模式上的关系的集合称为关系数据库

3.1.3 键

- 为了区分不同元组，用其中一个或多个属性值标识，能够唯一标识元组的属性或属性组称为关系的键
- 候选键：能够起标识作用，而其子集不能
 - 在一个关系中，如果有多个候选键，选其中的一个键作为主键 (primary key)
 - 若关系的键由多个属性组成，称为联合键
 - 关系的所有属性构成该关系的键，称为全键

3.1.4 完整性约束

- 关系模型的完整性规则是对关系的某种约束条件
- 为了维护数据库中数据与现实世界的一致性，对关系数据库的插入、删除和修改操作必须有一定的约束条件，这就是关系模型的三类完整性：
 - 实体完整性
 - ◆ 通常由关系系统自动支持
 - 参照完整性
 - ◆ 通常由关系系统自动支持
 - 用户定义的完整性
 - ◆ 反映应用领域需要遵循的约束条件，体现了具体领域中的语义约束
 - ◆ 用户定义后由系统支持

关系的两个**不变性**，应该由关系系统自动支持

3.1.4 完整性约束

1. 实体完整性约束 (Entity Integrity Constraint)

- 实体完整性是指主键的值不能为空或部分为空
- 实体完整性规则
 - ◆ 若属性（指一个或一组属性） A 是基本关系 R 的主属性，则属性 A 不能取空值
- 如果一个元组的键为空值，或部分为空，该元组将不可标识，不能表示任何实体，因而无意义

1. 实体完整性约束

□ 关系模型必须遵守实体完整性规则的原因

- (1) 实体完整性规则是针对基本关系而言的
 - ◆ 一个基本表通常对应现实世界的一个实体集或多对多联系
- (2) 现实世界中的实体和实体是可区分的，即它们具有某种唯一性标识
- (3) 相应地，关系模型中以主键作为唯一性标识
- (4) 主键中的属性即主属性不能取空值
 - ◆ 主属性取空值，说明存在某个不可标识的实体，即存在不可区分的实体，这与 (2) 相矛盾

□ 因此这个规则称为实体完整性

关系间的引用

□在关系模型中实体及实体间的联系都是用关系来描述的，自然存在着关系与关系间的引用

□[例] 学生实体、专业实体

学生 (学号, 姓名, 性别, 专业号, 年龄)

主键

专业 (专业号, 专业名)

主键

□学生关系引用了专业关系的主键 “专业号”

□学生关系中的 “专业号” 值必须是确实存在的专业的专业号

- 设 F 是基本关系 R 的一个或一组属性，但不是关系 R 的键（码）。如果 F 与基本关系 S 的主键 K_s 相对应，则称 F 是基本关系 R 的**外键（外码）**
- 基本关系 R 称为**参照关系** (Referencing Relation)，基本关系 S 称为**被参照关系** (Referenced Relation) 或**目标关系** (Target Relation)

学生关系 $\xrightarrow{\text{专业号}}$ 专业关系

- 说明：关系 R 和 S 不一定是不同的关系
- S 的主键 K_s 和 R 的外键 F 必须定义在同一个或一组域上
- 外键并不一定要与相应的主键同名

学生（学号，姓名，性别，专业号，年龄，班长）

2. 参照完整性约束

- 参照完整性约束 (Reference Integrity Constraint) 是对关系中作为**外键**的值的约束，规定：
 - 如果关系 $R1$ 中属性 A 是另一个关系 $R2$ 中的主键，则对于关系 $R1$ 中的任一个元组在属性 A 上的值或者为空值，或者为另一个关系 $R2$ 中某个元组的主键的值

2. 参照完整性约束

□ 学生关系中每个元组的“专业号”属性只取下面两类值：

- (1) 空值，表示尚未给该学生分配专业
- (2) 非空值，该值必须是专业关系中某个元组的“专业号”值，表示该学生不可能分配到不存在的专业中

□ 即学生关系中的某个属性的取值需要参照专业关系中的属性取值

专业号	专业名
01	信息
02	数学
03	计算机

专业关系

学号	姓名	性别	专业号	年龄
801	张三	女	01	19
802	李四	男	01	20
803	王五	男	01	20
804	赵六	女	02	20
805	钱七	男	02	19

学生关系

2. 参照完整性约束

□[例] 学生实体及其内部的领导联系（一对多）

学生（学号，姓名，性别，专业号，年龄，班长）

□ “班长” 属性值可以取两类值：

- (1) 空值，表示该学生所在班级尚未选出班长，或该学生本人即是班长
- (2) 非空值，这时该值必须是关系中某个元组的学号值

学号	姓名	性别	专业号	年龄	班长
801	张三	女	01	19	802
802	李四	男	01	20	
803	王五	男	01	20	802
804	赵六	女	02	20	805
805	钱七	男	02	19	

3. 用户定义的完整性

- 不同的关系数据库系统由于应用环境的不同，往往还需要一些特殊的约束条件，这就是**用户定义完整性** (User-defined Integrity)
- 用户定义的完整性是针对某一具体关系数据库的约束条件，反映某一具体应用所涉及的数据必须满足的语义要求
 - 例如，属性值根据实际需要，要具备一些约束条件：如选课关系中成绩不能为负数；某些数据的输入格式要有一些限制等
- 关系模型应该提供定义和检验这类完整性的机制，以使用统一的、系统的方法处理它们，**而不要由应用程序承担这一功能**



第3章 关系数据库

□ 3.1 关系模型的基本概念

□ 3.2 关系代数

□ 3.3 元组关系演算

□ 3.4 域关系演算

□ 3.5 小结

3.2 关系代数

□3.2.0 关系代数概述

□3.2.1 传统的集合运算

- 并、差、交、广义笛卡尔积

□3.2.2 专门的关系运算

- 选择、投影、连接、除法

□3.2.3 扩充的关系运算

- 属性重命名、外连接

□3.2.4 举例

□3.2.5 ISBL语言（略）

3.2.0 关系代数概述

1. 关系代数

- 一种抽象的查询语言，用对关系的运算来表达查询

2. 关系代数运算的三个要素

- 运算对象：关系
- 运算结果：关系
- 运算符

3.2.0 关系代数概述

3. 按运算符的不同，关系代数运算的分类：

- 传统的集合运算
 - ◆ 并、差、交、广义笛卡尔积
 - ◆ 把关系看成元组的集合，以元组作为集合中元素来进行运算，其运算是从关系的“水平”方向即行的角度进行的
- 专门的关系运算
 - ◆ 选择、投影、连接、除
 - ◆ 不仅涉及行运算，也涉及列运算，这种运算是为数据库的应用而引进的特殊运算

3.2 关系代数

□ 3.2.0 关系代数概述

□ 3.2.1 传统的集合运算

- 并、差、交、广义笛卡尔积

□ 3.2.2 专门的关系运算

- 选择、投影、连接、除法

□ 3.2.3 扩充的关系运算

- 属性重命名、外连接

□ 3.2.4 举例

□ 3.2.5 ISBL语言 (略)

1. 并 (Union)

□ R 和 S

- 具有相同的目 n (即两个关系都有 n 个属性)
- 相应的属性取自同一个域

□ $R \cup S$

- 仍为 n 目关系, 由属于 R 或属于 S 的元组组成

$$R \cup S = \{ t \mid t \in R \vee t \in S \}$$

A	B	C
$a1$	$b1$	$c1$
$a1$	$b2$	$c2$
$a2$	$b2$	$c1$

R

A	B	C
$a1$	$b2$	$c2$
$a1$	$b3$	$c2$
$a2$	$b2$	$c1$

S

A	B	C
$a1$	$b1$	$c1$
$a1$	$b2$	$c2$
$a1$	$b3$	$c2$
$a2$	$b2$	$c1$

$R \cup S$

2. 差 (Difference)

□ R 和 S

- 具有相同的目 n
- 相应的属性取自同一个域

□ $R-S$

- 仍为 n 目关系，由属于 R 而不属于 S 的所有元组组成

$$R-S = \{ t \mid t \in R \wedge t \notin S \}$$

A	B	C
$a1$	$b1$	$c1$
$a1$	$b2$	$c2$
$a2$	$b2$	$c1$

R

A	B	C
$a1$	$b2$	$c2$
$a1$	$b3$	$c2$
$a2$	$b2$	$c1$

S

A	B	C
$a1$	$b1$	$c1$

$R-S$

3. 交 (Intersection)

□ R 和 S

- 具有相同的目 n
- 相应的属性取自同一个域

□ $R \cap S$

- 仍为 n 目关系，由既属于 R 又属于 S 的元组组成

$$R \cap S = \{ t \mid t \in R \wedge t \in S \}, \quad R \cap S = R - (R - S)$$

A	B	C
$a1$	$b1$	$c1$
$a1$	$b2$	$c2$
$a2$	$b2$	$c1$

R

A	B	C
$a1$	$b2$	$c2$
$a1$	$b3$	$c2$
$a2$	$b2$	$c1$

S

A	B	C
$a1$	$b2$	$c2$
$a2$	$b2$	$c1$

$R \cap S$

4. 笛卡尔积

□ 关系 R 和 S 的笛卡尔积为 R 中所有元组和 S 中所有元组的拼接

□ R : n 目关系, k_1 个元组

□ S : m 目关系, k_2 个元组

□ $R \times S$

- 列: $(n+m)$ 列的元组的集合
 - ◆ 元组的前 n 列是关系 R 的一个元组
 - ◆ 后 m 列是关系 S 的一个元组
- 行: $k_1 \times k_2$ 个元组
 - ◆ $R \times S = \{ \widehat{t_r t_s} \mid t_r \in R \wedge t_s \in S \}$

4. 笛卡尔积

R

A	B	C
$a1$	$b1$	$c1$
$a1$	$b2$	$c2$
$a2$	$b2$	$c1$

S

A	B	C
$a1$	$b2$	$c2$
$a1$	$b3$	$c2$
$a2$	$b2$	$c1$

$R \times S$

A	B	C	A	B	C
$a1$	$b1$	$c1$	$a1$	$b2$	$c2$
$a1$	$b1$	$c1$	$a1$	$b3$	$c2$
$a1$	$b1$	$c1$	$a2$	$b2$	$c1$
$a1$	$b2$	$c2$	$a1$	$b2$	$c2$
$a1$	$b2$	$c2$	$a1$	$b3$	$c2$
$a1$	$b2$	$c2$	$a2$	$b2$	$c1$
$a2$	$b2$	$c1$	$a1$	$b2$	$c2$
$a2$	$b2$	$c1$	$a1$	$b3$	$c2$
$a2$	$b2$	$c1$	$a2$	$b2$	$c1$

3.2 关系代数

□3.2.0 关系代数概述

□3.2.1 传统的集合运算

- 并、差、交、广义笛卡尔积

□3.2.2 专门的关系运算

- 选择、投影、连接、除法

□3.2.3 扩充的关系运算

- 属性重命名、外连接

□3.2.4 举例

□3.2.5 ISBL语言（略）

3.2.2 专门的关系运算

□为了叙述上的方便，首先引入几个记号

(1) $R, t \in R, t[A_i]$

- 设关系模式为 $R(A_1, A_2, \dots, A_n)$ ，它的一个关系设为 R
- $t \in R$ 表示 t 是 R 的一个元组
- $t[A_i]$ 则表示元组 t 中相应于属性 A_i 的一个分量

(2) 若 $A = \{A_{i_1}, A_{i_2}, \dots, A_{i_k}\}$ ，其中 $A_{i_1}, A_{i_2}, \dots, A_{i_k}$ 是 $A_1, A_2, \dots, A_n$ 中的一部分，则 A 称为属性列或属性组

- $t[A] = (t[A_{i_1}], t[A_{i_2}], \dots, t[A_{i_k}])$ 表示元组 t 在属性列 A 上诸分量的集合
- $\bar{A}$ 则表示 $\{A_1, A_2, \dots, A_n\}$ 中去掉 $\{A_{i_1}, A_{i_2}, \dots, A_{i_k}\}$ 后剩余的属性组

3.2.2 专门的关系运算

(3) $t_r \hat{\smile} t_s$

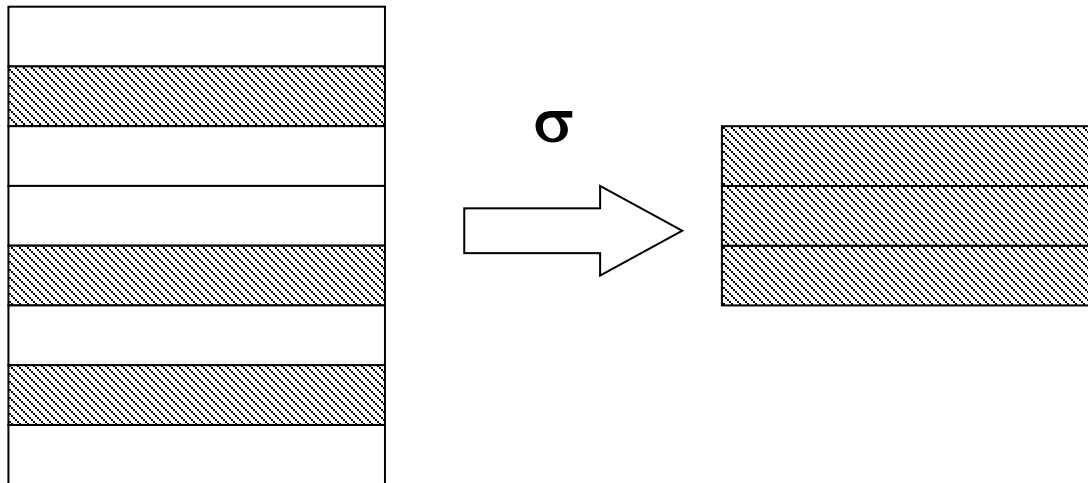
- R 为 n 目关系, S 为 m 目关系
- $t_r \in R, t_s \in S$, $t_r \hat{\smile} t_s$ 称为元组的连接。它是一个 $(n+m)$ 列的元组, 前 n 个分量为 R 中的一个 n 元组, 后 m 个分量为 S 中的一个 m 元组

1. 选择 (Selection)

□ 选择运算是关系上的一元运算，是从关系中选择满足一定条件的元组子集

$$\sigma_F(R) = \{ t \mid t \in R \wedge F(t) = \text{'真'} \}$$

- F 是限定条件的布尔表达式，由逻辑算符 ($\wedge$ 、 $\vee$ 、 $\neg$) 连接比较表达式组成
- 上式表示在关系 R 中选择使 $F(t)$ 为真的所有元组
- 选择运算是从行的角度进行的运算



1. 选择 (Selection)

[例] 查询信息系 (IS系) 全体学生

$\sigma_{Sdept='IS'}(Student)$

学号 Sno	姓名 Sname	性别 Ssex	年龄 Sage	所在系 Sdept
95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS

结果:

Sno	Sname	Ssex	Sage	Sdept
95002	刘晨	女	19	IS
95004	张立	男	19	IS

1. 选择 (Selection)

[例] 查询年龄小于20岁的学生

$\sigma_{\text{Sage} < 20}(\text{Student})$

学号 Sno	姓名 Sname	性别 Ssex	年龄 Sage	所在系 Sdept
95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS

结果:

Sno	Sname	Ssex	Sage	Sdept
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS

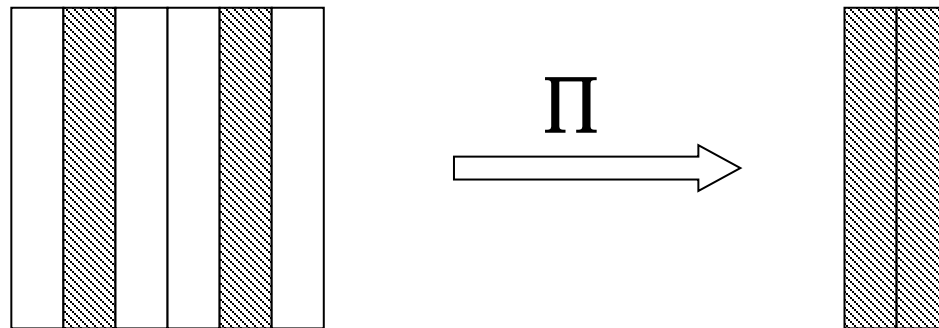
2. 投影 (Projection)

□ 在关系 R 上的投影运算表示为

$$\Pi_A(R) = \{t[A] \mid t \in R\}$$

- 其中, Π 是投影算符, A 是关系 R 属性的子集, $t[A]$ 表示 R 中元组在属性集 A 上的值, 或为元组 t 在 A 上的投影
- 从 R 中选择出若干属性列组成新的关系

□ 投影操作主要是从列的角度进行运算



□ 但投影之后不仅取消了原关系中的某些列, 而且还可能取消某些元组 (避免重复行)

2. 投影 (Projection)

[例] 查询学生的姓名和所在系，即求Student关系上学生姓名和所在系两个属性上的投影

$$\Pi_{\text{Sname, Sdept}}(\text{Student})$$

结果：

学号 Sno	姓名 Sname	性别 Ssex	年龄 Sage	所在系 Sdept
95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS

Sname	Sdept
李勇	CS
刘晨	IS
王敏	MA
张立	IS

3. 连接 (Join)

- 连接运算是把二个关系中的元组按条件连接起来，形成一个新关系
 - 条件连接
 - 自然连接
- 条件连接也称 θ 连接，是将二个关系中满足 θ 条件的元组拼接起来形成新元组的集合
 - 设属性 A 和 B 分别是关系 R 和 S 上的属性，且定义在同一个域上， R 和 S 的连接记为：

$$R \underset{A \theta B}{\bowtie} S = \{t \mid t = t_r \frown t_s, t_r \in R \wedge t_s \in S \wedge t_r[A] \theta t_s[B]\}$$

◆ 其中， $\bowtie$ 是连接符， $A\theta B$ 为连接条件， θ 是比较符

3. 连接 (Join)

R

<i>A</i>	<i>B</i>	<i>C</i>
a_1	b_1	5
a_1	b_2	6
a_2	b_3	8
a_2	b_4	12

S

<i>B</i>	<i>E</i>
b_1	3
b_2	7
b_3	10
b_3	2
b_5	2

$R \bowtie S$
 $C < E$

<i>A</i>	<i>R.B</i>	<i>C</i>	<i>S.B</i>	<i>E</i>
a_1	b_1	5	b_2	7
a_1	b_1	5	b_3	10
a_1	b_2	6	b_2	7
a_1	b_2	6	b_3	10
a_2	b_3	8	b_3	10

3. 连接 (Join)

□ 条件连接

- 从 R 和 S 的笛卡尔积 $R \times S$ 中选取 R 关系在 A 属性组上的值与 S 关系在 B 属性组上值满足比较条件的元组

$$R \bowtie_{A \theta B} S = \sigma_{A \theta B} (R \times S)$$

□ 最常用的连接是两个属性值的相等比较

- θ 为“=”的连接运算称为等值连接

3. 连接 (Join)

R

A	B	C
a_1	b_1	5
a_1	b_2	6
a_2	b_3	8
a_2	b_4	12

S

B	E
b_1	3
b_2	7
b_3	10
b_3	2
b_5	2

等值连接 $R \bowtie S$
 $R.B=S.B$

A	R.B	C	S.B	E
a_1	b_1	5	b_1	3
a_1	b_2	6	b_2	7
a_2	b_3	8	b_3	10
a_2	b_3	8	b_3	2

3. 连接 (Join)

□ 自然连接 (Natural Join)

- 自然连接是一种特殊的等值连接；它要求两个关系中进行比较的分量必须是相同的属性组，并且在结果中去掉重复的属性列

$$R \bowtie S = \{t \mid t = \overbrace{t_r}^{t_s[A]}, t_r \in R \wedge t_s \in S \wedge t_r[A] = t_s[A]\}$$

3. 连接 (Join)

R

A	B	C
a_1	b_1	5
a_1	b_2	6
a_2	b_3	8
a_2	b_4	12

S

B	E
b_1	3
b_2	7
b_3	10
b_3	2
b_5	2

等值连接 $R \bowtie S$
 $R.B = S.B$

A	R.B	C	S.B	E
a_1	b_1	5	b_1	3
a_1	b_2	6	b_2	7
a_2	b_3	8	b_3	10
a_2	b_3	8	b_3	2

自然连接 $R \bowtie S$

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2

3. 连接 (Join)

R

<i>A</i>	<i>B</i>	<i>C</i>
a_1	b_1	5
a_1	b_2	6
a_2	b_3	8
a_2	b_4	12

S

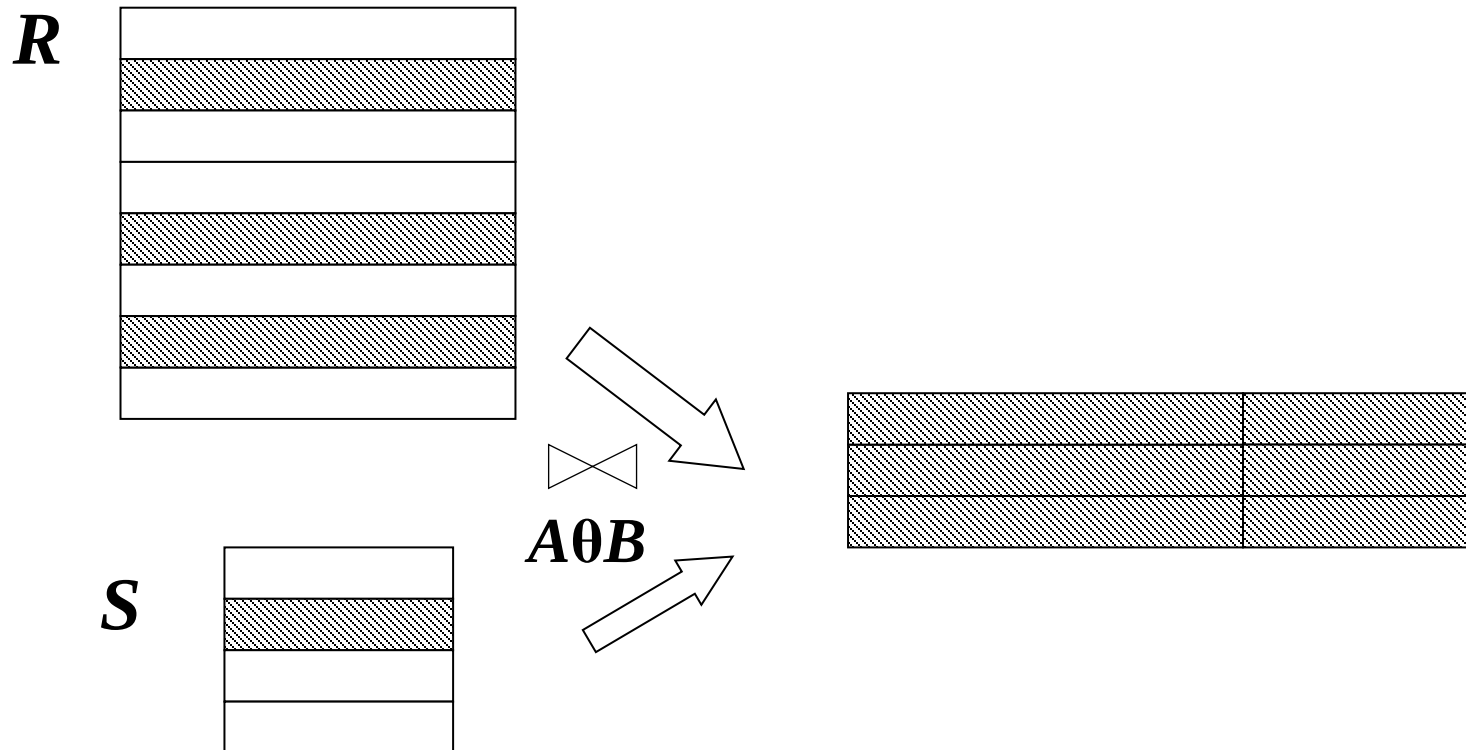
<i>B</i>	<i>C</i>	<i>E</i>
b_1	5	3
b_2	5	7
b_3	8	10
b_3	8	2
b_3	10	2

自然连接 $R \bowtie S$

<i>A</i>	<i>B</i>	<i>C</i>	<i>E</i>
a_1	b_1	5	3
a_2	b_3	8	10
a_2	b_3	8	2

3. 连接 (Join)

□一般的连接操作是从行的角度进行运算



□自然连接还需要取消重复列，所以是同时从行和列的角度进行运算

4. 除运算 (Division)

□ 除法运算是一个二元运算，用 $\div$ 表示

- 若 $R \div S$ ，要求 R 和 S 有定义在同一域上的属性或属性组
- $R \div S$ 的结果生成一个新关系 R' ， R' 的属性是 R 的属性中去掉与 S 具有公共域属性的其它属性

□ 设 $R(X, Y)$ ， $S(Y)$ ， $R'(X)$ 。则 $R \div S$ 记为：

$$R \div S = R' = \{ t \mid t \in R' \wedge t_r \in R \wedge t_s \in S \wedge t_r[R'] = t \wedge t \bowtie S \subseteq R \}$$

4. 除运算 (Division)

□ 结果集是 R 的属性中 **去掉与 S 具有公共域属性的其它属性**

□ 举例说明除法运算的含义

SC'

Sno	Cno
200701	C1
200701	C2
200701	C3
200702	C1
200702	C2
200703	C2
200703	C3

C'

Cno
C1

C''

Cno
C2
C3

$SC' \div C'$

Sno
200701
200702

选修了C1课的所有学生

同时选修了C2和C3课的所有学生

$SC' \div C''$

Sno
200701
200703

4. 除运算 (Division)

□ 除操作是同时从行和列角度进行运算

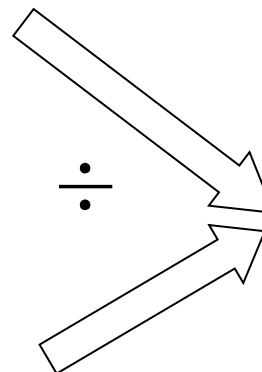
□ 除运算可用下式表示

$$R \div S = \Pi_X(R) - \Pi_X(\Pi_X(R) \times S - R)$$

- 上式中, X 为 R 中除去与 S 属性相同的其余属性

R

S



4. 除运算 (Division)

□[例] 设关系 R , S 分别如下, 求 $R \div S$

R

A	B	C
a_1	b_1	c_2
a_2	b_3	c_7
a_3	b_4	c_6
a_1	b_2	c_3
a_4	b_6	c_6
a_2	b_2	c_3
a_1	b_2	c_1

S

B	C	D
b_1	c_2	d_1
b_2	c_1	d_1
b_2	c_3	d_2

$R \div S$

A
a_1

4. 除运算 (Division)

□象集 Z_x ：给定一个关系 $R(X, Z)$, X 和 Z 为属性组。
 $t[X]=x$ 时, x 在 R 中的象集 (Images Set) 为：

$$Z_x = \{ t[Z] \mid t \in R, t[X]=x \}$$

它表示 R 中属性组 X 上值为 x 的诸元组在 Z 上分量的集合

□例：在关系 R 中,
 A 可以取四个值 $\{a_1, a_2, a_3, a_4\}$
 a_1 的象集为 $\{(b_1, c_2), (b_2, c_3), (b_2, c_1)\}$
 a_2 的象集为 $\{(b_3, c_7), (b_2, c_3)\}$
 a_3 的象集为 $\{(b_4, c_6)\}$
 a_4 的象集为 $\{(b_6, c_6)\}$

A	B	C
a_1	b_1	c_2
a_2	b_3	c_7
a_3	b_4	c_6
a_1	b_2	c_3
a_4	b_6	c_6
a_2	b_2	c_3
a_1	b_2	c_1

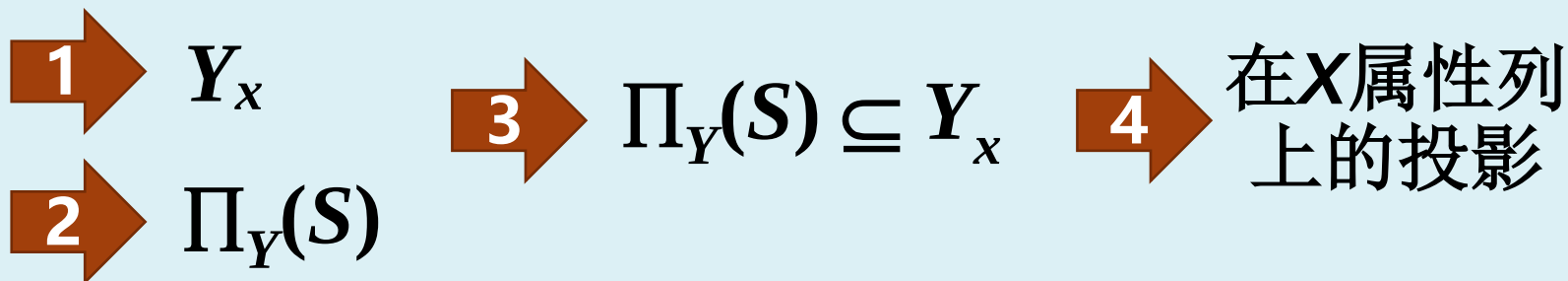
4. 除运算 (Division)

□ 给定关系 $R(X, Y)$ 和 $S(Y, Z)$, 其中 X, Y, Z 为属性组。 R 中的 Y 与 S 中的 Y 可以有不同的属性名, 但必须出自相同的域集。 R 与 S 的除运算得到一个新的关系 $P(X)$, P 是 R 中满足下列条件的元组在 X 属性列上的投影:

元组在 X 上分量值 x 的象集 Y_x 包含 S 在 Y 上投影的集合

$$R \div S = \{ t_r[X] \mid t_r \in R \wedge \Pi_Y(S) \subseteq Y_x \}$$

Y_x : x 在 R 中的象集, $x = t_r[X]$



4. 除运算 (Division)

□分析：在关系 R 中， A 可以取四个值 $\{a_1, a_2, a_3, a_4\}$

a_1 的象集为 $\{(b_1, c_2), (b_2, c_3), (b_2, c_1)\}$

a_2 的象集为 $\{(b_3, c_7), (b_2, c_3)\}$

a_3 的象集为 $\{(b_4, c_6)\}$

a_4 的象集为 $\{(b_6, c_6)\}$

S 在 (B, C) 上的投影为

$\{(b_1, c_2), (b_2, c_1), (b_2, c_3)\}$

只有 a_1 的象集包含了 S 在 (B, C) 属性组上的投影

所以 $R \div S = \{a_1\}$

S

B	C	D
b_1	c_2	d_1
b_2	c_1	d_1
b_2	c_3	d_2

R

A	B	C
a_1	b_1	c_2
a_2	b_3	c_7
a_3	b_4	c_6
a_1	b_2	c_3
a_4	b_6	c_6
a_2	b_2	c_3
a_1	b_2	c_1

$R \div S$

A
a_1

3.2.3 扩充的关系运算

1. 属性重命名

- 设 r 是模式 R 上的一个关系, A 是 R 中的一个属性
- B 为属性名, B 不是 R 中的属性, B 和 A 具有相同的域
- 设 $R'=(R-A)\cup B$, 则属性 A 被重命名为 B 后, 得到的关系 r' 记为:

$$r'(R') = \delta_{A \rightarrow B}(r)$$

- 重命名后的关系 r' 可表示如下:

$$r'(R') = \{ t' \mid t' \in r' \wedge t \in r \wedge t'[R-A] = t[R-A] \wedge t'[B] = t[A] \}$$

[例] 把学生关系中的学号Sno和姓名Sname重命名为Sno'和Sname'

$$\delta_{\text{Sno, Sname} \rightarrow \text{Sno}', \text{Sname}'}(\text{Student})$$

3.2.3 扩充的关系运算

1. 属性重命名 (续)

- 重命名运算可以同时的一组属性操作
- 通过属性重命名运算，可以
 - ◆ 在同一个关系上做自然连接运算
 - ◆ 做同一个关系的笛卡尔积
 - ◆ 将两个关系的等值连接方便地表示为自然连接

3.2.3 扩充的关系运算

2. 外连接

- 连接运算是把二个关系中的元组按条件连接起来，结果为满足条件的元组集合，这样的连接称为内连接 (inner join)，还有一种连接称为外连接
- 外连接 (outer join) 是对自然连接运算的扩展。外连接结果中除了满足连接条件的元组外还包含没有被连接的元组
- 左外连接
 - ◆ 左外连接的连接结果中包含了关系 R （左边关系）中不满足连接条件的元组，在这些元组对应关系 S 属性上的值为空值，记为： $R \bowtie_L S$

3.2.3 扩充的关系运算

2. 外连接 (续)

- 右外连接

- ◆ 右外连接的连接结果中包含了关系 S （右边关系）中不满足连接条件的元组，在这些元组对应关系 R 属性上的值为空值，记为： $R \bowtie_R S$

- 完全外连接

- ◆ 完全外连接的连接结果中包含了关系 R 中不满足连接条件的元组，同时也包含了关系 S 中不满足连接条件的元组。即连接结果是左外连接和右外连接结果的并，记为： $R \bowtie_F S$

左外连接



<i>R</i>	<i>A</i>	<i>B</i>	<i>C</i>
	<i>a</i> ₁	<i>b</i> ₁	5
	<i>a</i> ₁	<i>b</i> ₂	6
	<i>a</i> ₂	<i>b</i> ₃	8
	<i>a</i> ₂	<i>b</i> ₄	12

<i>S</i>	<i>B</i>	<i>C</i>	<i>E</i>
	<i>b</i> ₁	5	3
	<i>b</i> ₂	5	7
	<i>b</i> ₃	8	10
	<i>b</i> ₃	8	2
	<i>b</i> ₃	10	2

$R \bowtie_L S$

<i>A</i>	<i>B</i>	<i>C</i>	<i>E</i>
<i>a</i> ₁	<i>b</i> ₁	5	3
<i>a</i> ₁	<i>b</i> ₂	6	NULL
<i>a</i> ₂	<i>b</i> ₃	8	10
<i>a</i> ₂	<i>b</i> ₃	8	2
<i>a</i> ₂	<i>b</i> ₄	12	NULL

3.2.4 举例

□[例3-9] 检索计算机系学生的学号和姓名

Student

Sno	Sname	Sage	Ssex	Sdept
200701	刘明亮	18	男	计算机
200702	李和平	17	男	外语
200073	王 茵	21	女	计算机
200704	张小芳	20	女	数学

$\Pi_{\text{Sno, Sname}}(\sigma_{\text{Sdept}=\text{'计算机'}}(\text{Student}))$

3.2.4 举例

□[例] 查询选修了2号课程的学生们的学号

SC

学号Sno	课程号Cno	成绩Grade
2007001	1	92
2007001	2	85
2007001	3	88
2007002	2	90
2007002	3	80

$$\Pi_{Sno}(\sigma_{Cno='2'}(SC))$$
$$\{ '2007001', '2007002' \}$$

3.2.4 举例

□[例3-10] 检索选修了1号课程的学生姓名

首先考虑：从哪里取数据？

Student

Sno	Sname	Ssex	Sage	Sdept
95001	李勇	男	20	CS
95002	刘晨	女	19	IS
95003	王敏	女	18	MA
95004	张立	男	19	IS

SC

Sno	Cno	Grade
95001	1	92
95001	2	85
95001	3	88
95002	2	90
95002	3	80

3.2.4 举例

□[例3-10] 检索选修了1号课程的学生姓名

首先考虑：从哪里取数据？

Student ⋈ SC

Sno	Sname	Ssex	Sage	Sdept	Cno	Grade
95001	李勇	男	20	CS	1	92
95001	李勇	男	20	CS	2	85
95001	李勇	男	20	CS	3	88
95002	刘晨	女	19	IS	2	90
95002	刘晨	女	19	IS	3	80

其次考虑：符合什么条件的数据？ $\sigma_{Cno='1'}(Student \bowtie SC)$

最后，结果中都有哪些具体数据？

$\Pi_{Sname}(\sigma_{Cno='1'}(Student \bowtie SC))$

3.2.4 举例

□[例3-11] 检索不选C1课的学生信息

Student

Sno	Sname	Sage	Ssex	Sdept
200701	刘明亮	18	男	计算机
200702	李和平	17	男	外语
200073	王 茵	21	女	计算机
200704	张小芳	20	女	数学

SC

Sno	Cno	Grade
200701	C1	85
200701	C2	70
200701	C3	78
200702	C1	81
200702	C2	84
200703	C2	75
200703	C3	90

Student – ($\Pi_{Sno}(\sigma_{Cno='C1'}(SC)) \bowtie$ Student)

3.2.4 举例

□[例] 查询学生‘刘明亮’选修的全部课程的课程名

Sno	Sname	Sage	Ssex	Sdept
200701	刘明亮	18	男	计算机
200702	李和平	17	男	外语
200073	王 茵	21	女	计算机
200704	张小芳	20	女	数学

Student

Course

Cno	Cname	Cdept
C1	C语言	计算机
C2	英语	外语
C3	数据库	计算机
C4	数学	数学

SC

Sno	Cno	Grade
200701	C1	85
200701	C2	70
200701	C3	78
200702	C1	81
200702	C2	84
200703	C2	75
200703	C3	90

3.2.4 举例

□[例] 查询学生‘刘明亮’选修的全部课程的课程名

Sno	Sname		Sno	Cno	Grade	Cno	Cname	
200701	刘明亮	...	200701	C1	85	C1	C语言	...
200701	刘明亮	...	200701	C2	70	C2	英语	...
200701	刘明亮	...	200701	C3	78	C3	数据库	...
200702	李和平	...	200702	C1	81	C1	C语言	...
200702	李和平	...	200702	C2	84	C2	英语	...
200073	王 茵	...	200703	C2	75	C2	英语	...
200073	王 茵	...	200703	C3	90	C3	数据库	...

Student ⋈ SC ⋈ Course

3.2.4 举例

□[例] 查询学生‘刘明亮’选修的全部课程的课程名

首先考虑：从哪里取数据？

Student ⋈ SC ⋈ Course

其次考虑：符合什么条件的数据？

$\sigma_{\text{Sname}='刘明亮'}(\text{Student} \bowtie \text{SC} \bowtie \text{Course})$

最后，结果中都有哪些具体数据？

$\Pi_{\text{Cname}}(\sigma_{\text{Sname}='刘明亮'}(\text{Student} \bowtie \text{SC} \bowtie \text{Course}))$

3.2.4 举例

□[例] 查询至少选修了一门其直接先行课为5号课程的课程的学生姓名

Student

Sno	Sname	Sage	Ssex	Sdept
200701	刘明亮	18	男	计算机
...	...	...	...	...

SC

Sno	Cno	Grade
200701	C1	85
...	...	...

Course

课程号Cno	课程名Cname	先行课Cpno	学分Ccredit
C1	数据库	C5	4
C2	数学		2
C3	信息系统	C1	4
C4	操作系统	C6	3
C5	数据结构	C7	4
...	...	...	...

3.2.4 举例

□[例] 查询至少选修了一门其直接先行课为5号课程的课程的学生姓名

Sno	Sname		Sno	Cno	Grade	Cno	Cname	Cpno
200701	刘明亮	...	200701	C1	85	C1	数据库	C5
200701	刘明亮	...	200701	C2	70	C2	数学	
200701	刘明亮	...	200701	C3	78	C3	信息系统	C1
200702	李和平	...	200702	C1	81	C1	数据库	C5
200702	李和平	...	200702	C2	84	C2	数学	
200073	王 茵	...	200703	C2	75	C2	数学	
200073	王 茵	...	200703	C3	90	C3	信息系统	C1

$\Pi_{\text{Sname}}(\sigma_{\text{Cpno}='C5'}(\text{Student} \bowtie \text{SC} \bowtie \text{Course}))$

3.2.4 举例

□[例] 查询至少选修了一门其直接先行课为5号课程的课程的学生姓名

$$\Pi_{\text{Sname}}(\sigma_{\text{Cpno}='C5'}(\text{Student} \bowtie \text{SC} \bowtie \text{Course}))$$
$$\Pi_{\text{Sname}}(\sigma_{\text{Cpno}='C5'}(\text{Course}) \bowtie \text{SC} \bowtie \Pi_{\text{Sno}, \text{Sname}}(\text{Student}))$$
$$\Pi_{\text{Sname}}(\Pi_{\text{Sno}}(\sigma_{\text{Cpno}='C5'}(\text{Course}) \bowtie \text{SC}) \bowtie \Pi_{\text{Sno}, \text{Sname}}(\text{Student}))$$

3.2.4 举例

□[例] 查询选修了全部课程的学生号码和姓名

Student

Sno	Sname	Sage	Ssex	Sdept
200701	刘明亮	18	男	计算机
...	...	...	...	...

SC

Sno	Cno	Grade
200701	C1	85
...	...	...

Course

Cno	Cname	Cpno	Ccredit
C1	数据库	C5	4
...	...	...	...

$$\Pi_{Sno, Cno}(SC) \div \Pi_{Cno}(Course) \bowtie \Pi_{Sno, Sname}(Student)$$

3.2.4 举例

□[例] 查询至少选修1号课程和3号课程的学生学号

首先建立一个临时关系 K

Cno
1
3

然后求

$$\Pi_{\text{Sno}, \text{Cno}}(\text{SC}) \div K$$

于是：

$$\Pi_{\text{Sno}, \text{Cno}}(\text{SC}) \div K = \{2007001\}$$

SC

Sno	Cno	Grade
200701	1	85
200701	2	70
200701	3	78
200702	1	81
200702	2	84
200703	2	75
200703	3	90

3.2.4 举例

□[例3-13] 插入学号为200504的学生选修了C4课、成绩为88分的选课记录

$SCU\{ ('200504', 'C4', 88) \}$

□[例3-14] 删除学生‘刘明亮’选修的英语课

$SC-$

$(\Pi_{Sno}(\sigma_{Sname='刘明亮'}(Student))) \bowtie SC \bowtie \Pi_{Cno}(\sigma_{Cname='英语'}(Course))$

3.2.4 举例



□[例3-15] 检索与李勇在同一个系的学生的学号和姓名

Student

Sno	Sname	Ssex	Sage	Sdept
95001	李勇	男	20	CS
95002	刘晨	女	19	CS
95003	王敏	女	18	MA
95004	张立	男	19	IS

3.2.4 举例

□[例3-15] 检索与李勇在同一个系的学生的学号和姓名

Student $\bowtie$ $\delta_{Sno, Sname, Sage, Ssex \rightarrow Sno', Sname', Sage', Ssex'}(Student)$

Sno	Sname	Ssex	Sage	Sdept	Sno'	Sname'	Ssex'	Sage'
95001	李勇	男	20	CS	95001	李勇	男	20
95001	李勇	男	20	CS	95002	刘晨	女	19
95002	刘晨	女	19	CS	95001	李勇	男	20
95002	刘晨	女	19	CS	95002	刘晨	女	19
95003	王敏	女	18	MA	95003	王敏	女	18
95004	张立	男	19	IS	95004	张立	男	19

- 在对学生关系中的学号、姓名、年龄和性别重命名后，只有属性名Sdept是相同的，自然连接的结果为系相同的学生信息

3.2.4 举例

□[例3-15] 检索与李勇在同一个系的学生的学号和姓名

Student

Sno	Sname	Ssex	Sage	Sdept
95001	李勇	男	20	CS
95002	刘晨	女	19	CS
95003	王敏	女	18	MA
95004	张立	男	19	IS

- 在对学生关系中的学号、姓名、年龄和性别重命名后，只有属性名Sdept是相同的，自然连接的结果为系相同的学生信息

$$\Pi_{Sno', Sname'}(\sigma_{Sname='李勇'}(Student \bowtie \delta_{Sno, Sname, Sage, Ssex \rightarrow Sno', Sname', Sage', Ssex'}(Student)))$$

3.2 关系代数

□ 用关系代数运算可以完成对数据的检索、插入和删除操作

- 查询：查询的表达能力是其中最重要的部分

- ◆ 选择 (select), 投影 (project), 连接 (join), 除 (divide), 并 (union), 差 (difference), 交 (intersection), 笛卡尔积等

- 数据更新：插入 (insert), 删除 (delete), 修改 (update)

□ 关系操作的特点

- 集合操作方式，操作的对象和结果都是集合

- ◆ 一次一集合

- 非关系数据模型的数据操作方式

- ◆ 一次一记录

3.2 关系代数

□在关系代数运算中，并、差、笛卡儿积、选择、投影是基本的关系代数运算，其它的运算可以由这些基本运算表示。如：

- 交运算可用差运算表示： $R \cap S = R - (R - S)$
- 连接运算可由选择和笛卡儿积表示：

$$R \bowtie_{A \theta B} S = \sigma_{A \theta B} (R \times S)$$

- 除运算可用下式表示：

$$R \div S = \Pi_X(R) - \Pi_X(\Pi_X(R) \times S - R)$$

◆ 上式中， X 为 R 中除去与 S 属性相同的其余属性

3.2 关系代数

□ 关系代数运算

- 关系代数运算

- ◆ 并、差、交、笛卡尔积、选择、投影、连接、除

- 基本运算

- ◆ 并、差、笛卡尔积、选择、投影

- 交、连接、除

- ◆ 可以用5种基本运算来表达

- ◆ 引进它们并不增加语言的能力，但可以简化表达

□ 关系代数表达式

- 关系代数运算经有限次复合后形成的式子



第3章 关系数据库

- 3.1 关系模型的基本概念
- 3.2 关系代数
- 3.3 元组关系演算
- 3.4 域关系演算
- 3.5 小结

关系数据语言的分类

□ 关系代数语言

- 用对关系的运算来表达查询要求
- 典型代表：ISBL

□ 关系演算语言：用谓词来表达查询要求

- 按谓词变元的基本对象的不同分为：

◆ 元组关系演算语言

- 谓词变元的基本对象是元组变量
- 典型代表：APLHA, QUEL

◆ 域关系演算语言

- 谓词变元的基本对象是域变量
- 典型代表：QBE

元组关系演算语言ALPHA



□由E. F. Codd提出

- INGRES所用的QUEL语言是参照ALPHA语言研制的，这一语言并没有实际实现

□ALPHA语言的基本格式为：

操作语句 <工作空间名> (表达式): 操作条件

- 表达式用于指定语句的操作对象，它可以是关系名或(和) 属性名，一条语句可以同时操作多个关系或多个属性
- 操作条件是一个逻辑表达式，用于将操作结果限定在满足条件的元组中；操作条件可以为空
- 除此之外，还可以在基本格式的基础上加上排序要求，定额要求等

□检索语句 GET

- 例：查询所有被选修的课程号码

GET W (SC.Cno)

- 例：查询所有学生的数据

GET W (Student)

- 例：查询信息系 (IS) 中年龄小于20的学生学号和年龄

GET W (Student.Sno, Student.Sage): Student.Sdept='IS' $\wedge$
Student.Sage<20

- 例：查询计算机系 (CS) 学生的学号、年龄，结果按年龄降序排序

GET W (Student.Sno, Student.Sage): Student.Sdept='CS'
DOWN Student.Sage

元组关系演算语言ALPHA



□更新语句

- PUT, HOLD, UPDATE, DELETE, DROP

□修改操作步骤

- ① 用HOLD语句将要修改的元组从数据库中读到工作空间中

HOLD 工作空间名 (表达式1) [: 操作条件]

HOLD语句是带上并发控制的GET语句

- ② 用宿主语言修改工作空间中元组的属性
- ③ 用UPDATE语句将修改后的元组送回数据库中

UPDATE 工作空间名



修改操作



□[例] 把95007学生从计算机系转到信息系

HOLD W (Student.Sno, Student.Sdept):

Student.Sno='95007'

(从Student关系中读出95007学生的数据)

MOVE 'IS' TO W.Sdept

(用宿主语言进行修改)

UPDATE W

(把修改后的元组送回Student关系)



插入操作



□ 步骤

- ① 用宿主语言在工作空间中建立新元组
- ② 用PUT语句把该元组存入指定关系中

PUT 工作空间名 (关系名)

□ PUT语句只对一个关系操作

□[例] 学校新开设了一门2学分的课程‘计算机组织与结构’，其课程号为8，直接先行课为6号课程。插入该课程元组

MOVE ‘8’ TO W.Cno

MOVE ‘计算机组织与结构’ TO W.Cname

MOVE ‘6’ TO W.Cpno

MOVE ‘2’ TO W.Ccredit

PUT W (Course)



删除操作



□ 步骤

- ① 用**HOLD**语句把要删除的元组从数据库中读到工作空间中
- ② 用**DELETE**语句删除该元组

DELETE 工作空间名

[例] 95110学生因故退学，删除该学生元组

HOLD W (Student): Student.Sno='95110'

DELETE W

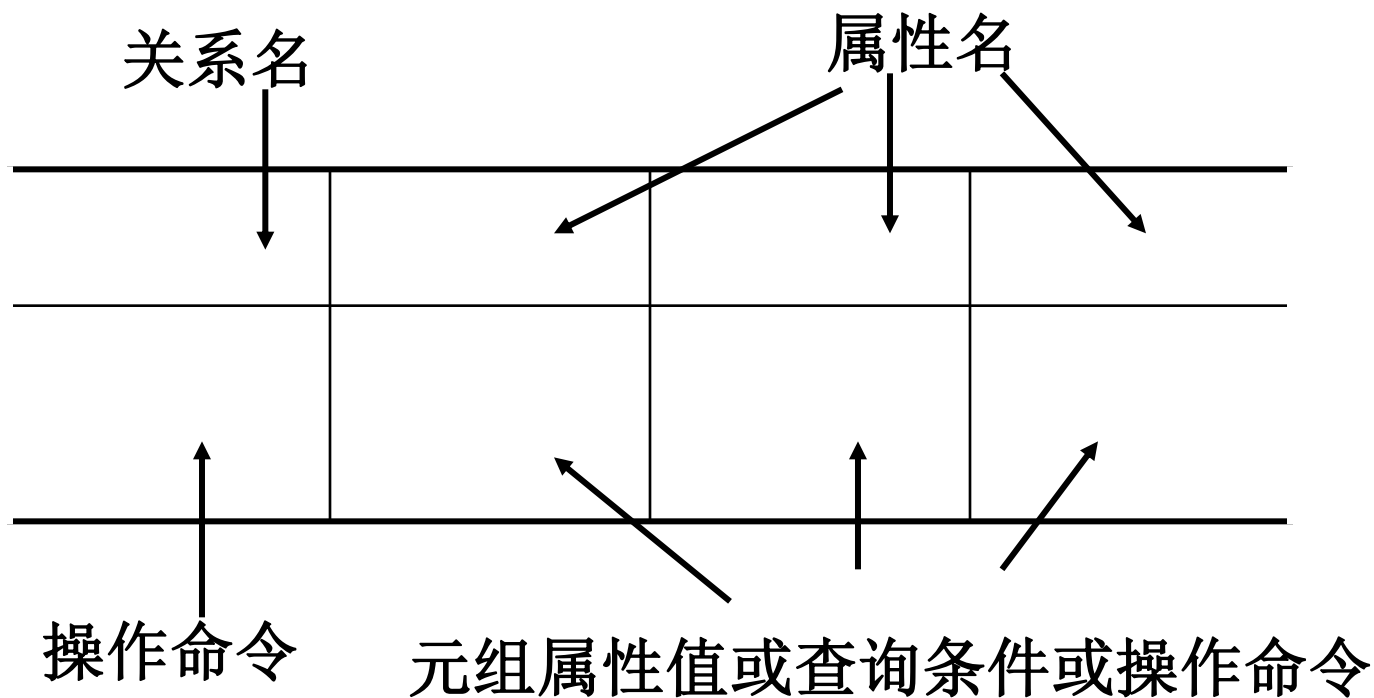
□ 一种典型的域关系演算语言

- 由M. M. Zloof提出
- 1978年在IBM370上得以实现
- QBE也指此关系数据库管理系统

□ QBE: Query By Example

- 基于屏幕表格的查询语言
- 查询要求: 以填写表格的方式构造查询
- 用示例元素 (域变量) 来表示查询结果可能的情况
- 查询结果: 以表格形式显示

□QBE操作框架





检索操作



(1) 用户提出要求

(2) 屏幕显示空白表格

(3) 用户在最左边一栏输入要查询的关系名，例如
Student

Student					

(4) 系统显示该关系的属性名

Student	Sno	Sname	Ssex	Sage	Sdept

(5) 用户在上面构造查询要求

Student	Sno	Sname	Ssex	Sage	Sdept
		P. <u>T</u>			IS

(6) 屏幕显示查询结果

Student	Sno	Sname	Ssex	Sage	Sdept
		P. <u>李勇</u> 张立			IS

□[例] 把95001学生的年龄改为18岁

方法(1): 将操作符“U.”放在值上

Student	Sno	Sname	Ssex	Sage	Sdept
	95001			U. 18	

方法(2): 将操作符“U.”放在关系上

Student	Sno	Sname	Ssex	Sage	Sdept
U.	95001			18	

码95001标明要修改的元组。“U.” 标明所在的行是修改后的新值。由于主码是不能修改的，**所以系统不会混淆要修改的属性**



修改操作



□[例] 把95001学生的年龄增加1岁

Student	Sno	Sname	Ssex	Sage	Sdept
U.	95001			<u>17</u>	
	95001			<u>17+1</u>	

分两行分别表示改前和改后的示例元素

必须将操作符“U.”放在关系上

□[例] 把信息系女生95701，姓名张三，年龄17岁存入数据库中

Student	Sno	Sname	Ssex	Sage	Sdept
I.	95701	张三	女	17	IS



删除操作



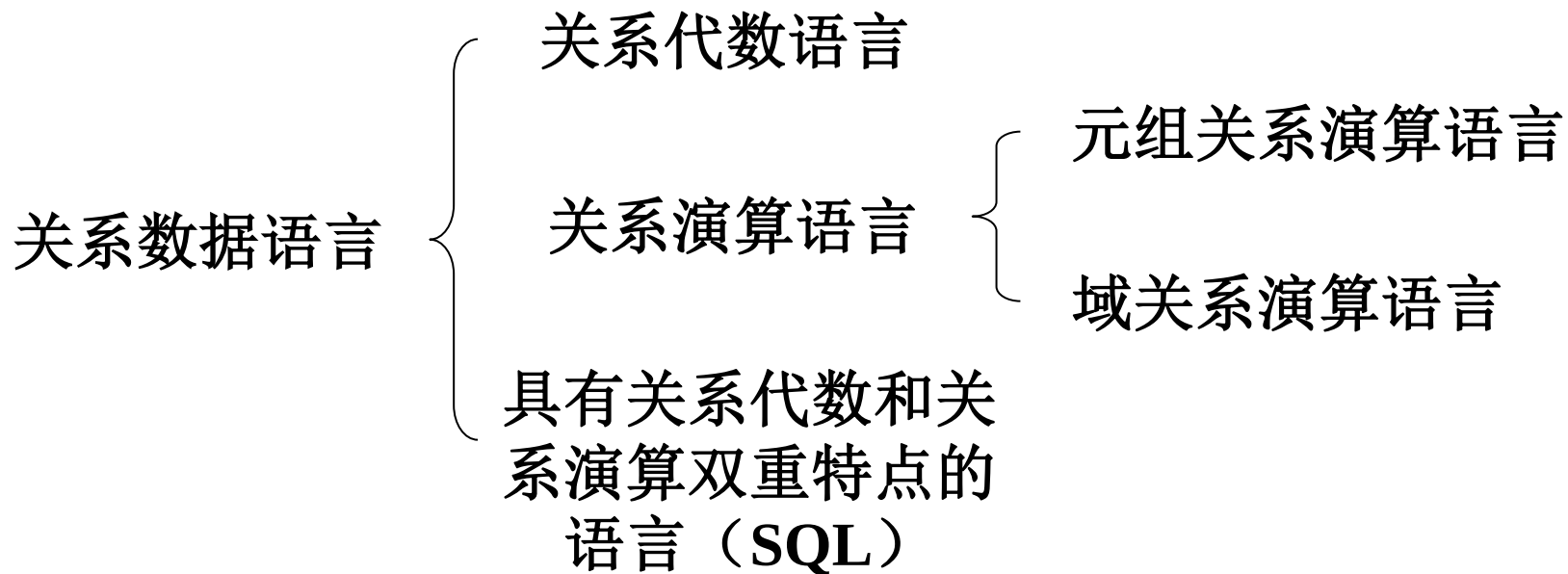
□[例] 删除学生95089

Student	Sno	Sname	Ssex	Sage	Sdept
D.	95089				

为保证参照完整性，删除95089学生前，先删除95089学生选修的全部课程

SC	Sno	Cno	Grade
D.	95089		

关系数据语言的分类



□具有关系代数和关系演算双重特点的语言

- 典型代表：SQL (Structured Query Language)
- 它是集Query、DDL、DML、DCL于一体的关系数据语言，充分体现了关系数据语言的特点和优点，是关系数据库的标准语言

关系数据语言的特点

□关系数据语言的特点

- 是一种高度非过程化的语言
- 存取路径的选择由DBMS的优化机制来完成
- 用户不必用循环结构就可以完成数据操作
- 能够嵌入高级语言中使用
- 关系代数、元组关系演算和域关系演算三种语言在表达能力上完全等价

3.4.3 关系运算的安全限制

1. 关系运算的安全限制

- 关系是集合论中的概念，在集合论中，关系可以是无限的
- 但在关系数据库中，关系被限定为是有限的
- 不产生无限关系的表达式称为安全运算表达式，所采取的措施称安全限制
 - ◆ 关系代数运算是安全的，只要参加运算的关系是有限的，关系运算的结果关系也是有限的。关系运算的有限次复合不会出现无限关系
 - ◆ 关系演算不一定是安全的，需要加以限制

2. 三种关系运算的等价性

- 可以证明，在加以安全限制后，这三种关系运算在表达关系的功能上是等价的

3.5 小结

关系模型三要素

□ 关系数据结构

- 关系、关系模式、关系数据库

□ 关系操作

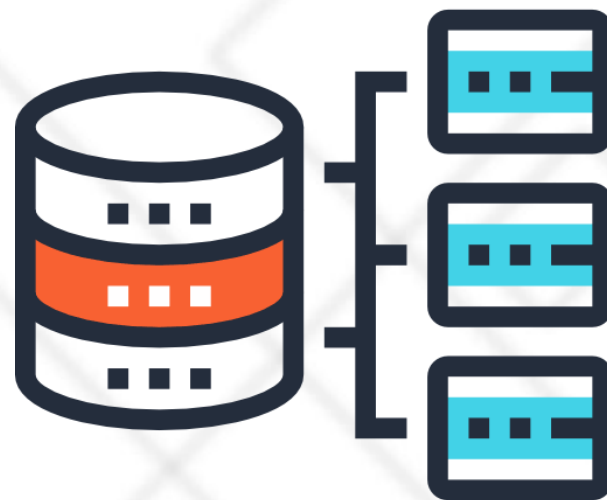
- 关系代数：是关系数据库系统中实现关系操作的一种语言
- 关系演算

□ 关系的完整性

- 实体完整性
- 参照完整性
- 用户定义的完整性



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY



作业

《数据库系统原理》第3章 习题4